Some *yarn add* commands

yarn add <package …> [--exact/-E]: Using *–exact* or *-E* installs the exact version of the package. The default is to use the most recent release with the same major version.

yarn add <package …> [--tilde/-T] : Using *–tilde* or *-T* installs the most recent release of the packages that have the same minor version. The default is to use the most recent release with the same major version. For example, *yarn add debug@1.2.3 –tilde* would accept *1.2.9* but not *1.3.0*.

- ***yarn add package-name@tag:*** This command is used to install a package of a specified tag, e.g., beta, next or latest.
- ***yarn add <package…> [--dev/-D]:*** Using *–dev* or *-D* will install one or more packages in *devDependencies* in *package.json*.
  By default, all these packages get installed from the npm registry, but we can specify a local folder path, URL, gzip tarball local file path, Git repository URL, etc. A few examples are given below.
- ***yarn add package-name:*** Installs the package from the npm registry unless we specify another one in *package.json*.
- ***yarn add <file:/local/local/folder>:*** Installs a package that is on your local file system. This is useful to test out other packages of yours that haven't been published to the public/ private registry yet.
- ***yarn add <file:/local/foder/tarball.tgz>***: Installs a package from a gzipped tarball, which could be used to share a package before publishing it.
- ***yarn add <git remote url>:*** Installs a package from a remote Git repository.
- ***yarn add <git remote url>#<branch/commit/tag>:*** Installs a package from a remote Git repository at a specific Git branch, Git commit or Git tag.
- ***yarn add <https://my-project.org/package.tgz>:*** Installs a package from a remote gzipped tarball.
  In case you are using npm, you would use *–save* or *--save-dev.* In Yarn, these have been replaced by *yarn add* and *yarn add –dev.*

Some *yarn install* commands

- ***yarn install –check-files:*** Verifies that files already installed in *node_modules* are not removed.
- ***yarn install –force:*** Re-fetches all the packages, even ones that were previously installed.
- ***yarn install –ignore-scripts:*** Does not execute any scripts defined in the project *package.json* and its dependencies.
- ***yarn install –modules-folder <path>:*** By default, packages get installed in the project *node_modules* directory. With this command, you can specify a different path to install all dependencies.
- ***yarn install –no-lockfile:*** By default, for every installation, Yarn makes an entry in the *yarn.lock* file. This command instructs Yarn to neither read nor generate a *yarn.lock* lockfile.
- ***yarn install –production[true|false]:*** Yarn will not install any package listed in *devDependencies* if the *NODE_ENV* environment variable is set to *production.* Use this flag to instruct Yarn to ignore *NODE_ENV,* and to take its production-or-not status instead.
- ***yarn install –offline:*** Runs *yarn install* in offline mode.

Managing dependencies

Upgrading or deleting packages will automatically update *package.json* and *yarn.lock* files. Other developers working on the project can run *yarn install* to sync their own *node_modules* directories with the updated set of dependencies.

When we remove a package, it gets removed from *prod, dev* dependencies.

```
yarn remove [package-name]
ex: yarn remove mongoose
```

Packages can also be upgraded to the latest or a lower version.

```
yarn upgrade [package]
yarn upgrade [package]@[version]
yarn upgrade [package]@[tag]
```

Other useful Yarn commands

Yarn provides rich sets of commands, but I will explain only some of them.

After installing Node.js and Yarn, we can start using the Yarn commands to manage dependencies in our projects.

yarn init: This is the first command we should run to create the *package.json* file, which is used to manage information like the project's name, version or licence information, as well as the author's and contributors' names – basically, the details of the most important project dependencies. This command walks us through an interactive session to create a *package.json* file.

yarn config commands: Here is a list of a few of these.

- *yarn config list:* Displays the current configuration.
- *yarn config set <key> <value> [-g| --global]:* Sets the *config key* to a certain value.
- *yarn config get <key>:* Echoes the value for a given *key* to *stdout.*
- *yarn config delete <key>:* Deletes a given *key* from the *config*.

yarn cache commands: These list, clean and change the cache directory.

- *yarn cache ls:* Yarn stores every package in a global cache in your user directory on the file system. This command will print out every cached package.
- *yarn cache dir:* This command will print out the path where Yarn's global cache is currently stored.
- *yarn cache clean:* This will clear the global cache. It will be populated again the next time *yarn* or *yarn install* is